

Busca aleatória e AutoML

Escolher hiperparâmetros quando não dá para testar tudo, e o que a automação faz com isso (Aula 09)

O curso, semana a semana (1/2)

Aula (Dia/Mês)	Assunto	Evento
1 — 04/08	Introdução: fundamentos da avaliação de modelos	
2 — 11/08	Métricas de classificação I (confusão, precisão, recall, F1)	Lançamento do projeto
3 — 18/08	Métricas de classificação II (ROC-AUC, PR, log-loss)	
4 — 25/08	Métricas de regressão (MAE, RMSE, R ² , resíduos)	Projeto: proposta
5 — 01/09	Validação hold-out e Pipeline	
6 — 08/09	Validação cruzada I (k-fold, estratificado)	
7 — 15/09	Validação cruzada II (grupos, temporal, aninhada)	
8 — 22/09	Busca exaustiva de hiperparâmetros (grid search)	
9 — 06/10	Busca aleatória e menção a AutoML	Desafio 3 (subst.)

O curso, semana a semana (2/2)

Aula (Dia/Mês)	Assunto	Evento
10 — 13/10	Curvas de validação	
20/10	SEMANA ACADEMICA (SEM AULA)	
11 — 27/10	Curvas de aprendizado	
12 — 03/11	Seleção e comparação de modelos e Limiar de decisão e calibração	
13 — 10/11	Target Leakage e reprodutibilidade	
14 — 17/11	Seleção de atributos	
15 — 24/11	Estudo de caso integrado e Comunicação e reprodutibilidade dos resultados	Desafio 6 (subst.)
16 — 01/12	Revisão integrada e apresentações	Apresentação final

De onde viemos: a conta da Aula 08, e a de hoje

A floresta aleatória do lab = muitas árvores de decisão (Aula 01), cada uma treinada numa amostra diferente dos dados; a previsão é o voto delas.

AULA 08 · logística	valores	nº
C	0,1 · 1 · 10	3
class_weight	None · balanced	2
combinações	3 × 2	6
treinos (5 dobras)	6 × 5	30

POR QUE FOI BARATO

Só **2 hiperparâmetros**, com poucos valores cada. O "2" da conta é o nº de VALORES do class_weight, não o nº de hiperparâmetros.

HOJE · floresta aleatória	o que controla	valores na grade (exemplo)	nº
n_estimators	quantas árvores a floresta tem (mais árvores = média mais estável, treino mais lento)	50 · 100 · 150 · 200	4
max_depth	quantas perguntas em sequência cada árvore pode fazer (a complexidade)	2 · 4 · 6 · 8 · 12	5
min_samples_leaf	o mínimo de músicas em cada folha, o fim de cada caminho da árvore (impede folha que decora 1 exemplo)	1 · 5 · 10 · 15	4
max_features	a fração das variáveis sorteada a cada pergunta da árvore (deixa as árvores diferentes)	0,3 · 0,5 · 0,7 · 0,9	4
ccp_alpha	a poda: corta os galhos que melhoram pouco	0,0001 · 0,001 · 0,01 · 0,1	4

$$4 \times 5 \times 4 \times 4 \times 4 = 1.280 \text{ combinações}$$
$$1.280 \times 4 \text{ dobras (as do lab)} = 5.120 \text{ treinos}$$

Com 5 dobras, como na Aula 08, seriam 6.400. Hoje: em vez de testar todas, SORTEAR algumas — sorteando 12 combinações (n_iter = 12), a mesma floresta custa 12 × 4 = 48 treinos.

Objetivo da aula

Saber **por que sortear combinações funciona** tão bem quanto testar todas, usar o **RandomizedSearchCV** com as distribuições certas, reportar a nota **sem se enganar** e reconhecer o que o **AutoML** automatiza e o que ele não automatiza.

O CAMINHO DE HOJE

1. **Por que sortear funciona** — o relevo das músicas, grade × aleatória e a conta de probabilidade.
2. **RandomizedSearchCV** — distribuições, faixas, código e leitura da saída.
3. **Refinar e reportar** — do grosso ao fino, e a nota honesta.
4. **Buscas com memória e AutoML** — bayesiana e automação do pipeline inteiro.

A ANALOGIA DE HOJE

Achar o cume de uma serra na neblina. Você só enxerga onde está, e cada leitura do altímetro custa uma hora de caminhada.

Grade: medir num quadriculado fixo. **Aleatória:** medir em pontos sorteados. **Bayesiana:** escolher o próximo ponto olhando as leituras anteriores.

O caso de hoje são as músicas dos labs 08 e 09: prever o "hit" (12% das faixas), com AP como régua. Cada leitura do altímetro = uma validação cruzada.

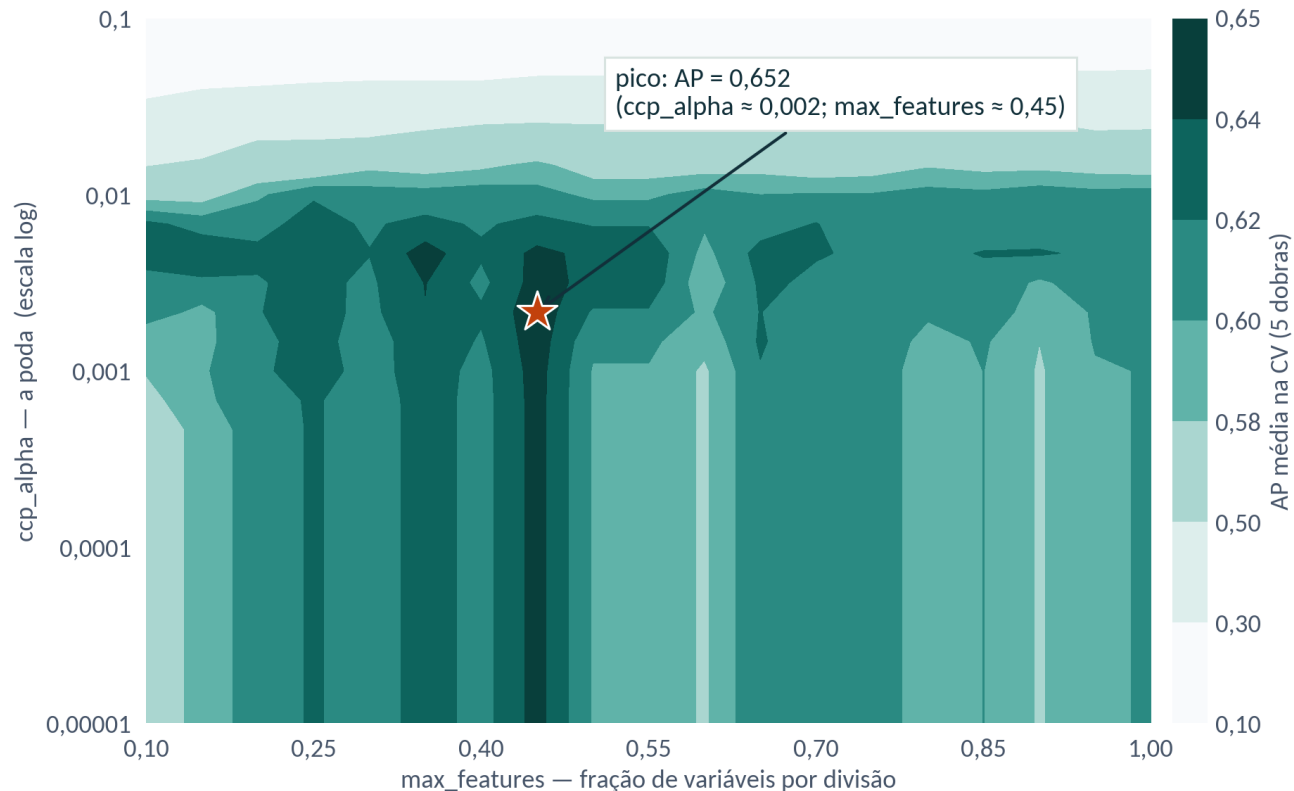
PARTE 1

Por que sortear funciona

O relevo das músicas, grade × aleatória no mesmo orçamento, e a conta que explica o resultado.

O problema: achar o pico sem ver o mapa

A floresta do lab nas músicas, variando 2 dos seus hiperparâmetros (slide 3). Cada cor = AP média de 5 dobras. 475 combinações $\times$ 5 = 2.375 treinos.



O QUE O MAPA ENSINA

Cada ponto custa 5 treinos (uma validação cruzada) e não há fórmula para ir direto ao pico: só medindo. Este mapa inteiro ninguém faz na prática.

A poda manda: sozinha, leva a AP de 0,12 a 0,65 — com `ccp_alpha` acima de 0,01 a floresta é podada demais e desaba.

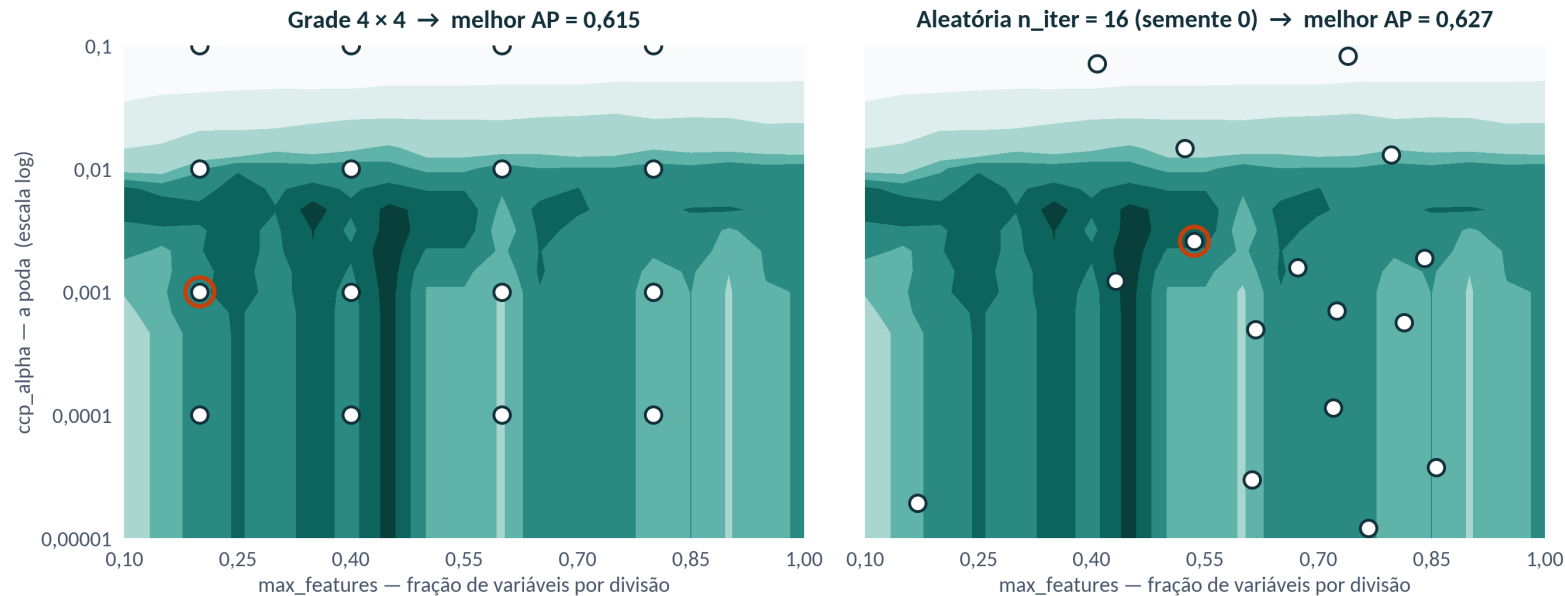
max_features muda pouco: na melhor poda, a AP só vai de 0,59 a 0,65. As listras são ruído.

A região boa é larga: com `ccp_alpha` abaixo de ~0,01, quase tudo passa de 0,58.

A pergunta da aula: com 16, 40 ou 60 medições em vez de 2.375, onde coloco cada uma?

Mesmo orçamento, dois desenhos: 16 medições no relevo das músicas

Os pontos brancos são as medições de cada busca, sobre o mapa do slide anterior. O círculo laranja é a melhor de cada uma.



O QUE ACONTECEU

Grade: só 4 valores de poda, e 2 deles (0,01 e 0,1) na região ruim: metade das 16 medições desperdiçada. Melhor: AP 0,615.

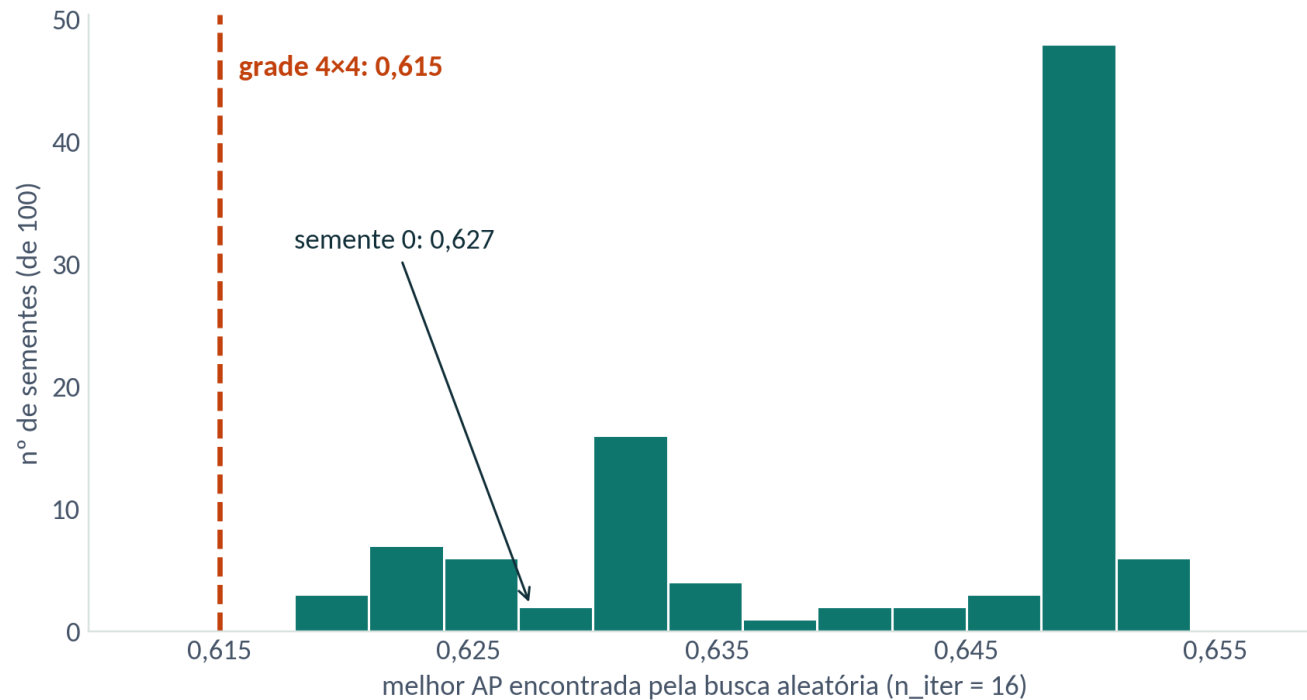
Aleatória (semente 0): 16 valores distintos de cada hiperparâmetro; 4 caíram acima de 0,01. Melhor: 0,627.

Aqui a aleatória ganhou. Sorte da semente 0? O próximo slide responde.

A grade repete cada valor de poda 4 vezes — inclusive os dois ruins. A aleatória nunca repete um valor.

Uma semente não diz nada: repetindo o sorteio 100 vezes

A mesma busca aleatória ($n_iter = 16$) com 100 sementes diferentes. A linha laranja é a grade 4×4 .



O QUE AS 100 SEMENTES MOSTRAM

100 de 100 buscas aleatórias ficaram acima da grade (0,615).

Média: 0,640. Pior caso: 0,618. Melhor: 0,654. A semente 0 (0,627) ficou até abaixo da média.

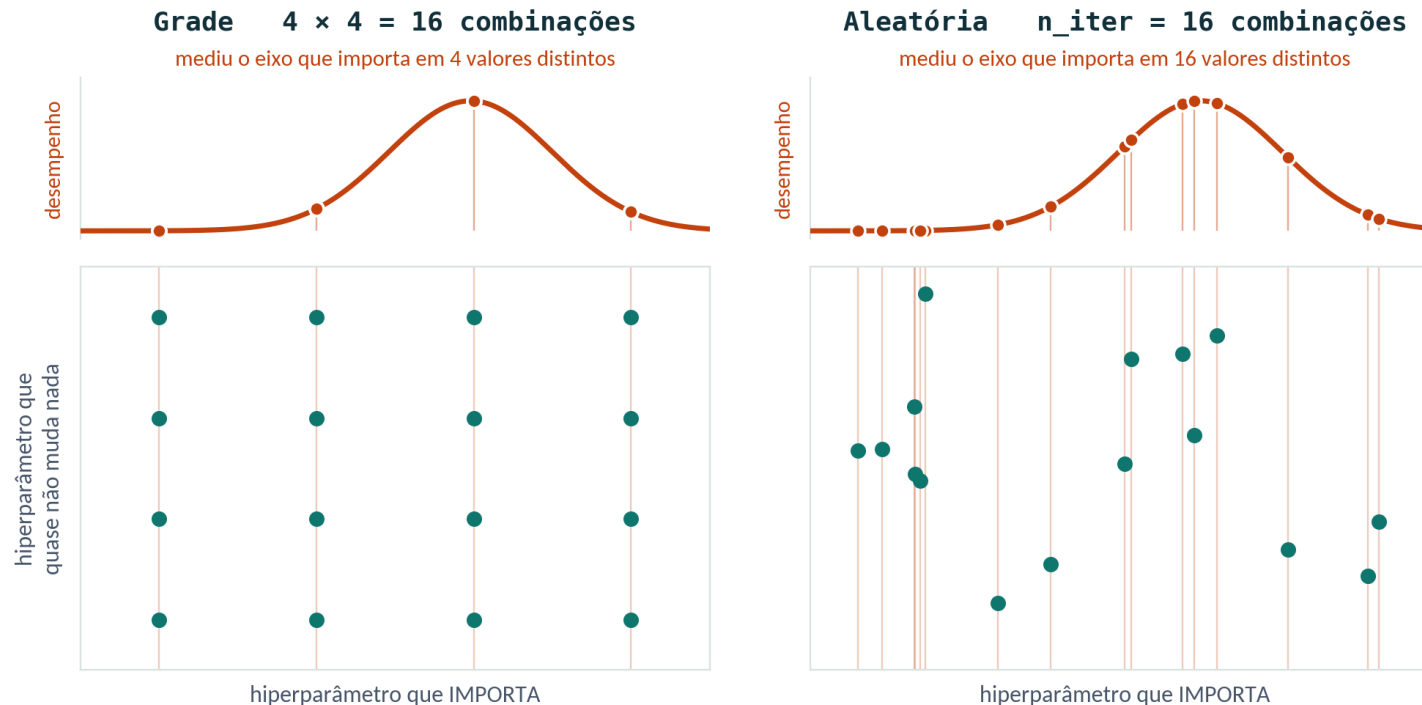
A promessa é de PROBABILIDADE: quase sempre boa, nunca garantida. A pior semente ficou só 0,003 acima da grade.

E a grade? É uma aposta só: os 4 valores de poda que você escolheu são tudo o que ela vai ver.

Por isso se fixa `random_state`: para reproduzir. Mas não se escolhe a semente pelo resultado; isso seria outra forma de "espiar o teste".

Por que o sorteio vence quando poucos hiperparâmetros importam

O argumento clássico de Bergstra e Bengio (2012), em 16 pontos.



O MECANISMO

Projete os pontos no eixo que **importa** (como a poda nas músicas).

Grade: os 16 pontos caem em só 4 valores, repetidos 4 vezes. Os 12 extras testaram o eixo que não importa.

Aleatória: cada ponto tem um valor NOVO em cada eixo, então são 16 leituras distintas do eixo que importa.

Com 5 hiperparâmetros e só 1 importante, a grade desperdiça ainda mais.

Nas músicas é exatamente assim: a poda manda e `max_features` muda pouco. E em geral você não sabe de antemão quem manda.

A conta que explica: $1 - (1 - p)^n$

Com n sorteios, qual a chance de pelo menos um cair na região boa, se ela ocupa uma fração p do espaço?

$$P(\text{nenhum cai}) = (1 - p)^n \quad \rightarrow \quad P(\text{pelo menos um cai na região boa}) = 1 - (1 - p)^n$$

n_iter	região boa = 5% do espaço	músicas: AP $\geq 0,62 = 15\%$
10	$1 - 0,95^{10} = 40\%$	$1 - 0,85^{10} = 81\%$
16	$1 - 0,95^{16} = 56\%$	$1 - 0,85^{16} = 93\%$
30	$1 - 0,95^{30} = 79\%$	$1 - 0,85^{30} = 99\%$
60	$1 - 0,95^{60} = 95\%$	$\approx 100\%$

O QUE A CONTA DIZ

O nº de hiperparâmetros não aparece na conta: vale com 2 ou com 20. Na grade, cada hiperparâmetro novo multiplica o custo.

Nas músicas, 16 sorteios dão 93%. E das 100 sementes, 97 passaram de 0,62 — a conta acertou.

Região estreita pede mais orçamento: com 5%, são 60 sorteios para 95%. É assim que se escolhe o n_iter .

A ressalva: "p" é a fração do espaço que VOCÊ definiu. Faixa mal escolhida encolhe a região boa — é o assunto da Parte 2.

PARTE 2

RandomizedSearchCV

Distribuições no lugar de listas, faixas bem escolhidas e a leitura da saída.

De listas para distribuições

Na grade você lista valores; na aleatória você diz DE ONDE sortear. As distribuições vêm do `scipy.stats`, a biblioteca de estatística do Python.

distribuição (scipy.stats)	sorteia	use para	exemplo
<code>randint(a, b)</code>	inteiros de a até b – 1, todos com a mesma chance	contagens	<code>max_depth: randint(2, 15)</code>
<code>uniform(início, largura)</code>	reais em [início, início + largura]	frações e proporções	<code>max_features: uniform(0.3, 0.6)</code>
<code>loguniform(a, b)</code>	reais de a até b, com a mesma chance em cada ORDEM DE GRANDEZA	o que varia por fator de 10	C: <code>loguniform(1e-2, 1e2)</code>
lista [..]	um item da lista, todos com a mesma chance	categorias	<code>class_weight: [None, "balanced"]</code>

ATENÇÃO À ASSINATURA DO `uniform`

`uniform(0.3, 0.6)` sorteia entre 0,3 e **0,9** (início + largura), e não entre 0,3 e 0,6. É o erro mais comum na montagem do dicionário.

No código, `1e-2` = 0,01 e `1e2` = 100 (notação científica: o número depois do "e" é a potência de 10).

Exemplo na mão: por que o C se sorteia em escala log

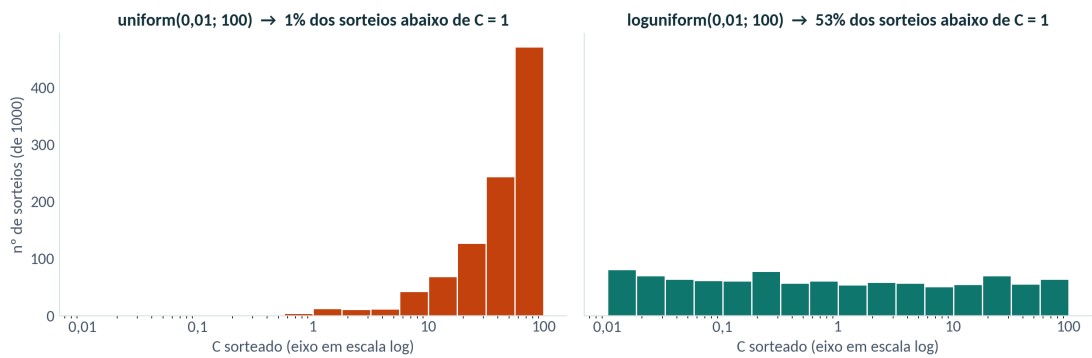
O C da regressão logística (Aula 08), na faixa [0,01; 100]: são 4 ordens de grandeza. Onde caem os sorteios?

faixa de C	uniform: chance	loguniform: chance
0,01 a 0,1	0,09%	25%
0,1 a 1	0,9%	25%
1 a 10	9%	25%
10 a 100	90%	25%

A CONTA

uniform: a chance é proporcional ao TAMANHO do trecho. O trecho de 10 a 100 tem 90 das 99,99 unidades: 90%.

loguniform: sorteia o EXPOENTE uniformemente entre -2 e 2. Cada ordem de grandeza recebe 1/4.



Nas músicas a poda boa fica abaixo de 0,01. Num uniform(0; 0,1), 90% dos sorteios cairiam acima de 0,01 — na região onde a floresta desaba.

A faixa também é decisão sua

Faixa larga demais desperdiça orçamento; estreita demais deixa o pico de fora.

O QUE ACONTECEU NA SEMENTE 0

Faixa usada: `ccp_alpha` em `loguniform(0,00001; 0,1)`, ou seja, 4 ordens de grandeza.

A região ruim (acima de 0,01) ocupa 1 dessas 4: **25% dos sorteios** em média.

Na semente 0 foram **4 de 16** — exatamente o esperado. É o preço de não saber a escala de antemão.

Com a faixa de 0,00001 a 0,01 (3 ordens), todo o orçamento teria ido para a região útil.

COMO ESCOLHER A FAIXA

Comece largo e em log quando não souber a escala: é barato descobrir onde fica a região boa.

Use o que já se sabe: documentação, defaults e resultados anteriores (o default da floresta é `ccp_alpha = 0`, sem poda).

Olhe onde os melhores caíram. Se eles encostam na borda da faixa, o pico pode estar FORA: amplie.

Depois estreite (Parte 3): a segunda rodada gasta o orçamento só onde vale a pena.

O sorteio só é esperto dentro do espaço que você deu a ele. Definir o espaço continua sendo trabalho de cientista de dados.

RandomizedSearchCV na prática: as músicas

Cada argumento é uma peça de uma aula anterior.

```
from scipy.stats import loguniform, uniform
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([("esc", StandardScaler()),          # Aula 05: sem vaziar
                  ("clf", RandomForestClassifier(random_state=0))])
dist = {"clf__ccp_alpha": loguniform(1e-5, 1e-1),    # a poda, em log
        "clf__max_features": uniform(0.1, 0.8)}      # de 0,1 a 0,9
cv = StratifiedKFold(5, shuffle=True, random_state=0) # Aulas 06-07
busca = RandomizedSearchCV(pipe, dist, n_iter=40, cv=cv,      # orçamento: 40 × 5
                           scoring="average_precision",      # Aulas 03-04
                           random_state=0, n_jobs=-1)         # reprodutível; usa todos os núcleos
busca.fit(X_tr, y_tr)                                         # o teste fica guardado
```

O RESULTADO NAS MÚSICAS

`best_score_` = **0,652** com `ccp_alpha` $\approx$ 0,0019 e `max_features` $\approx$ 0,45 — o pico do mapa. Custo: $40 \times 5 = 200$ treinos, contra 2.375 do mapa inteiro.

O prefixo "clf__" aponta para o passo "clf" do Pipeline, exatamente como na grade da Aula 08.

Lendo a saída: o pódio e o ruído

cv_results_ ordenado pelo rank: as 5 primeiras da busca com n_iter = 16 (semente 0).

rank	ccp_alpha	max_features	AP média	± desvio (5 dobras)
1	0,0026	0,54	0,627	0,118
2	0,0005	0,62	0,618	0,104
2	0,00003	0,61	0,618	0,104
4	0,0012	0,43	0,616	0,112
5	0,0007	0,72	0,613	0,112

COMO LER

Os 5 primeiros empatam: diferença de 0,014; desvio de ±0,11.

É o platô (Aula 08): o 1º lugar ganhou em parte por sorte.

Os dois 2º são o mesmo modelo: 0,62 e 0,61 de 18 variáveis dão 11 por pergunta, e as duas podas quase não cortam.

```
res = pd.DataFrame(busca.cv_results_).sort_values("rank_test_score")
res[["param_clf__ccp_alpha", "param_clf__max_features", "mean_test_score", "std_test_score"]].head()
```

Na dúvida, prefira a configuração mais simples ou mais estável (menor desvio). best_params_ é só a primeira linha desta tabela.

PARTE 3

Refinar e reportar

Do grosso ao fino, e a nota que se pode prometer.

Grade ou aleatória? Na prática, as duas, em sequência

PREFIRA A GRADE QUANDO

Poucos eixos discretos — 2 ou 3 hiperparâmetros, com poucos valores cada.

Você quer cobrir TUDO — o espaço é pequeno e cabe inteiro no orçamento.

Está refinando — uma grade pequena em volta de uma região já conhecida.

PREFIRA A ALEATÓRIA QUANDO

Espaço grande ou contínuo — muitos hiperparâmetros, ou valores reais.

Orçamento fixo — você decide o custo (n_{iter}), e não a grade.

Não sabe quem importa — o sorteio testa valores novos em todos os eixos.

A RECEITA DO GROSSO AO FINO (coarse-to-fine)

1. **Aleatória larga, em log**, para descobrir onde fica a região boa.
2. **Grade pequena** em volta dos melhores pontos, para confirmar e refinar.
3. **Pare quando os números virarem platô**: refinar dentro do ruído é gastar orçamento à toa.

É o mapa da neblina: primeiro saltos longos para achar a encosta certa, depois passos curtos perto do cume.

Exemplo: o refino nas músicas

Grade 4 × 4 em volta do melhor ponto da aleatória (ccp_alpha ≈ 0,0019, max_features ≈ 0,45).

ccp_alpha \ max_features	0,30	0,45	0,60	0,75
0,00095	0,610	0,649	0,577	0,612
0,0019	0,620	0,652	0,595	0,608
0,0038	0,625	0,649	0,583	0,614
0,0076	0,620	0,628	0,607	0,607

O QUE O REFINO MOSTROU

Todas as 16 células ficaram entre 0,577 e 0,652.

A melhor (0,652) é o próprio ponto que a aleatória já tinha achado: o refino não encontrou nada melhor. E as diferenças cabem no desvio de ±0,12.

Conclusão honesta: é um platô. A busca já tinha achado a região boa; refinar mais não ajuda.

Na Aula 10 (curvas de validação) vamos olhar um hiperparâmetro de cada vez para enxergar esse platô com clareza.

Buscar sem se enganar: o best_score_ é otimista

Quem escolhe não mede (Aula 07). Três números da mesma busca nas músicas (n_iter = 40).

número	valor	de onde vem	serve para
best_score_	0,652	o MÁXIMO de 40 médias de CV, na mesma validação que escolheu	escolher os hiperparâmetros
nested CV (5 × 5)	0,615 ± 0,126	a busca inteira refeita em 5 rodadas; cada rodada mede numa dobra que a busca não viu	estimar o desempenho do PROCEDIMENTO
teste guardado	0,711	o best_estimator_ tocando o teste uma única vez (Aula 05) — só 38 hits: uma medição com muito ruído	a confirmação final

```
externo = StratifiedKFold(5, shuffle=True, random_state=1)
notas = cross_val_score(busca, X_tr, y_tr, cv=externo,
                        scoring="average_precision") # a busca
```

POR QUE O best_score_ SOBE

É o máximo de 40 notas ruidosas (maldição do vencedor, Aula 07). O teste, 0,711, cai dentro da faixa das notas externas.

Notas externas da nested: 0,635 · 0,623 · 0,793 · 0,626 · 0,398. Com tanta dispersão, reporte média ± desvio — um número só engana.

PARTE 4

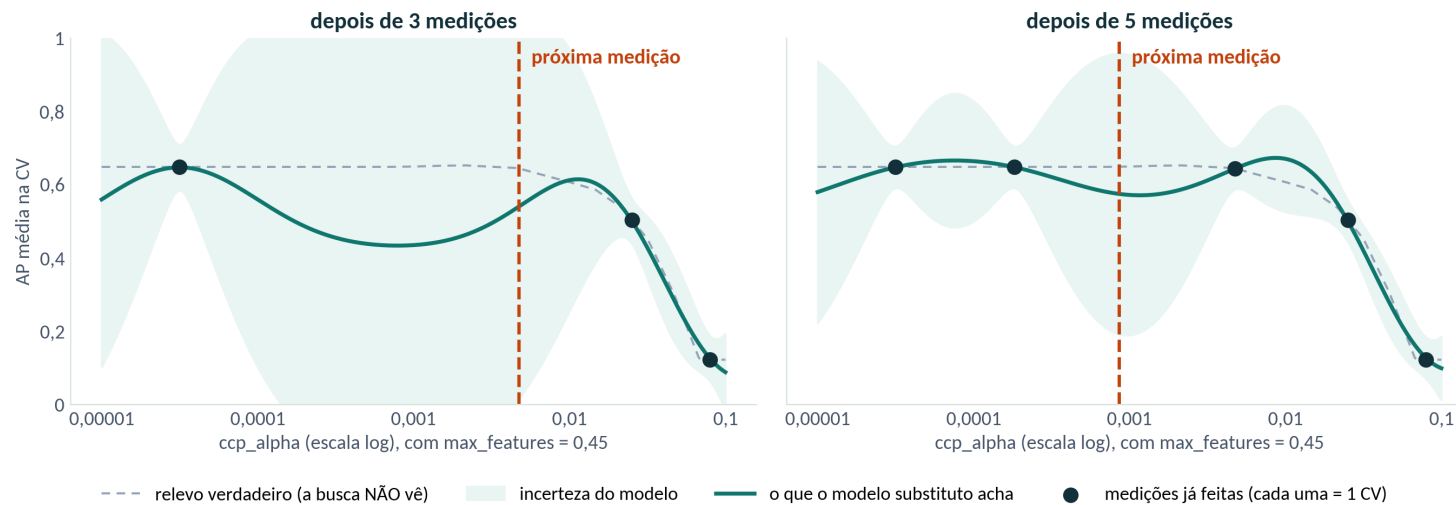
Buscas com memória e AutoML

O que vem depois da aleatória, e até onde a automação vai.



Otimização bayesiana: a busca que aprende com as medições

Grade e aleatória não têm memória: a 40ª tentativa ignora as 39 anteriores. A bayesiana usa todas.



COMO ELA DECIDE

1. Modelo substituto: com as medições já feitas, ajusta um modelo barato de "poda → AP", com incerteza (a faixa clara).

2. Onde medir: onde o modelo prevê nota alta OU onde ele está muito incerto.

Esquerda: com 3 medições, há um buraco enorme entre 0,00003 e 0,025. Ela vai EXPLORAR ali, onde pode haver algo melhor.

Direita: com 5 medições, as boas estão todas abaixo de 0,01. Ela vai APROVEITAR: medir entre elas (perto de 0,001).

Mesmo relevo das músicas (um corte com max_features = 0,45). Ferramentas: Optuna, scikit-optimize. Em detalhe, só em Ciência de Dados 2.

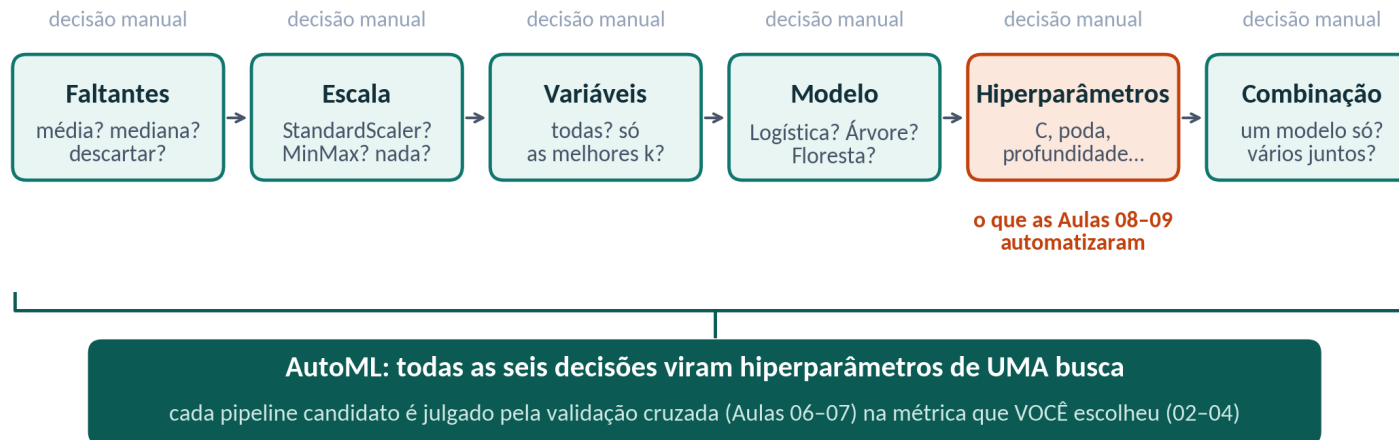
Três jeitos de gastar o mesmo orçamento

	grade	aleatória	bayesiana
onde mede	quadriculado fixo	pontos sorteados	onde o modelo substituto indica
memória	nenhuma	nenhuma	usa todas as medições anteriores
dá para rodar as tentativas ao mesmo tempo?	sim, todas de uma vez	sim, todas de uma vez	pouco: cada ponto depende dos anteriores
o custo cresce com o nº de hiperparâmetros?	sim, multiplica	não: $n_iter \times \text{dobras}$	não: $n_iter \times \text{dobras}$
brilha quando	espaço pequeno e discreto	espaço grande, poucos eixos importam	cada avaliação é MUITO cara (horas)
no scikit-learn	GridSearchCV	RandomizedSearchCV	fora do sklearn (Optuna, skopt)

A bayesiana economiza tentativas, mas cada uma espera a anterior. Com tentativas rápidas e um computador com vários núcleos, a aleatória costuma bastar.

AutoML: e se TODAS as decisões virassem hiperparâmetros?

Até aqui automatizamos uma etapa do pipeline. O AutoML automatiza a montagem inteira.



A IDEIA

O gargalo: tratar faltantes, escalar, escolher variáveis e escolher o modelo continuam sendo decisões manuais, no mesmo Pipeline da Aula 05.

O AutoML trata cada decisão como mais um eixo do espaço de busca. Na literatura, isso se chama CASH: escolha conjunta de algoritmo e hiperparâmetros.

O motor é o de hoje: sorteio ou bayesiana, com validação cruzada julgando cada pipeline.

Ferramentas: auto-sklearn, TPOT, FLAML, H2O. Todas são uma busca muito grande sobre pipelines inteiros.

O que o AutoML NÃO automatiza

O motor é validação cruzada. Tudo o que está fora dela continua sendo decisão sua.

A MÁQUINA FAZ

Testa milhares de pipelines com validação cruzada, em paralelo.

Escolhe pré-processamento, modelo e hiperparâmetros.

Registra tudo: cada tentativa fica num log, o que dá rastreabilidade e reprodutibilidade.

VOCÊ AINDA DECIDE

O problema e a métrica: AP ou acurácia? Quanto custa cada erro? (Aulas 02–04)

O esquema de validação: estratificado, por grupo, temporal? (Aulas 06–07)

O que pode entrar no modelo: nas músicas, a coluna popularidade dá AP 0,936 contra 0,721, porque ela praticamente define o hit (vazamento, Aula 01). O AutoML adoraria usá-la.

A nota honesta: mais tentativas deixam o best_score_ mais otimista. A nested CV ou o teste guardado continuam obrigatórios.

A tecnologia navega o relevo; quem desenha o mapa (dados, métrica, validação) é você.

Erros comuns e boas práticas

ERROS COMUNS

Sortear em escala linear o que varia por ordem de grandeza (C, ccp_alpha).

Faixa larga demais, que gasta o orçamento na região ruim, ou estreita demais, que deixa o pico de fora.

n_iter minúsculo diante do espaço: a conta $1 - (1 - p)^n$ não fecha.

Buscar fora do Pipeline: o pré-processamento vê a dobra de validação e vaza.

Reportar o best_score_ como resultado final, ou escolher a semente pelo resultado.

Deixar o AutoML usar uma coluna que vaza o alvo.

BOAS PRÁTICAS

loguniform para o que varia por fator de 10; randint para contagens; listas para categorias.

Aleatória larga primeiro; grade pequena para confirmar; pare no platô.

Escolha o n_iter pela conta de probabilidade, não no chute.

Sempre a busca sobre o Pipeline, com o scoring da métrica que importa.

Reporte a nested CV (média $\pm$ desvio) ou o teste guardado; fixe o random_state.

Revise as variáveis antes de automatizar qualquer coisa.

Na próxima (Aula 10): curvas de validação. Um hiperparâmetro de cada vez, treino $\times$ validação, para diagnosticar sub e sobreajuste.